

UNIVERSIDAD DE GUADALAJARA



CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

INGENIERÍA EN COMPUTACIÓN

ARQUITECTURA DE COMPUTADORAS

MIPS

Fase 3

Nombres:

- Luis Giovanni Rodríguez Hernandez
- Sara Iriel Alonso Jaquez
- Leonel Octavio Rios Loreto
- Jorge Abraham Moran Encina

Fecha: 23 de Mayo de 2026

1. Introducción

La arquitectura MIPS (Microprocessor without Interlocked Pipeline Stages) es una arquitectura tipo RISC (Reduced Instruction Set Computer) que sirve para ejecutar instrucciones simples en una cantidad reducida de ciclos de reloj. Permitiendo así una implementación modular y eficiente mediante un conjunto reducido de instrucciones y un modelo basado en registros.

Este procesador organiza la ejecución mediante un conjunto de etapas que permiten obtener un mayor rendimiento utilizando segmentación o pipeline. Las principales etapas son: búsqueda de instrucción (Instruction Fetch), decodificación (Instruction Decode), ejecución (Execute), acceso a memoria (Memory Access) y escritura de resultados (Write Back).

Durante las fases anteriores del proyecto se implementó el soporte para instrucciones tipo R y tipo I. En esta tercera fase se añadieron los módulos necesarios para soportar instrucciones tipo J, específicamente la instrucción de salto incondicional. Además, se incorporaron buffers entre etapas para preparar el datapath para funcionamiento segmentado mediante pipeline.

El pipeline consiste en dividir el procesamiento de instrucciones en diferentes etapas que pueden ejecutarse simultáneamente. Para lograr una sincronización adecuada se utilizan registros intermedios llamados buffers, cuya función es almacenar temporalmente la información entre etapas y actualizarla mediante flancos de reloj.

2. Marco Teórico

Las instrucciones MIPS de este proyecto tiene una longitud fija de 32 bits, lo que facilita la decodificación y simplifica el diseño del hardware. Estas instrucciones se dividen principalmente en tres formatos:

Instrucciones Tipo R (Register):

Realizan operaciones entre registros. Utilizan campos para registros fuente, registro destino y función específica.

Instrucciones Tipo I (Immediate):

Utilizan valores inmediatos o permiten acceso a memoria y bifurcaciones.

Instrucciones Tipo J (Jump):

Permiten modificar el flujo normal del programa mediante saltos a una dirección específica.

La estructura general de las instrucciones implementadas en el proyecto es:

Tipo R					
Opcode	Rs	Rt	Rd	Shamnt	Funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Tipo I			
Opcode	Rs	Rt	Inmediato
6 bits	5 bits	5 bits	16 bits

Tipo j	
Opcode	Dirección
6 bits	26 bits

Pipeline

El pipeline es una técnica utilizada en arquitecturas de procesadores para incrementar el rendimiento mediante la división de la ejecución de instrucciones en diferentes etapas. Esto permite que varias instrucciones puedan procesarse al mismo tiempo. Al usar Pipeline permite aumentar el rendimiento del procesador debido a que cada etapa puede trabajar simultáneamente con diferentes instrucciones.

MergeSort

El algoritmo seleccionado para el proyecto fue **MergeSort**, el cual pertenece a la categoría de algoritmos de ordenamiento basados en la estrategia *Divide y Vencerás (Divide and Conquer)*. Su funcionamiento consiste en dividir repetidamente un arreglo en partes más pequeñas hasta obtener subarreglos de un solo elemento y posteriormente combinar dichos subarreglos de manera ordenada.

El proceso del algoritmo puede dividirse en tres etapas principales:

- Dividir el arreglo original en dos partes.

- Aplicar recursivamente el algoritmo a cada subarreglo.
- Combinar los arreglos ordenados mediante una etapa de mezcla.

La ventaja principal de MergeSort es que mantiene un tiempo de ejecución eficiente incluso para conjuntos grandes de datos, siendo más estable que otros algoritmos de ordenamiento simples.

Descripción del funcionamiento

El algoritmo MergeSort trabaja mediante la estrategia de dividir y vencer. Su funcionamiento consiste en separar el arreglo principal en secciones más pequeñas hasta reducir el problema a partes simples. Posteriormente se realiza una etapa de combinación donde los elementos son comparados y reorganizados de manera ordenada hasta obtener el arreglo completamente ordenado.

Para la implementación dentro del procesador MIPS desarrollado, fue necesario adaptar el algoritmo al conjunto reducido de instrucciones soportadas por el datapath implementado, utilizando operaciones aritméticas, acceso a memoria y saltos para controlar el flujo de ejecución.

Descripción de módulos implementados

Unidad de Control (UNITCTRL)

Este módulo se encarga de generar las señales necesarias para controlar cada una de las etapas del datapath. Dependiendo del opcode recibido, activa señales como RegWrite, MemRead, MemWrite, Branch y Jump.

ALU y Control de ALU

La ALU realiza operaciones aritméticas y lógicas necesarias durante la ejecución de instrucciones. El módulo ALUCTRL genera la señal de selección correspondiente dependiendo del tipo de operación solicitada.

Banco de registros

Permite almacenar y recuperar datos utilizados durante la ejecución. Se implementaron dos puertos de lectura y un puerto de escritura.

Contador de programa y lógica de salto

El PC almacena la dirección actual de ejecución y fue modificado para permitir el soporte de instrucciones tipo J mediante multiplexores y lógica adicional de cálculo de dirección.

3. Objetivos

Objetivo General

El objetivo general es diseñar e implementar un datapath MIPS de 32 bits que pueda ejecutar instrucciones tipo R, tipo I y tipo J mediante Verilog, incorporando buffers para soportar una arquitectura segmentada mediante pipeline. .

Objetivos Específicos

- Implementar soporte para instrucciones tipo J dentro del datapath.
- Diseñar la lógica necesaria para el cálculo de direcciones de salto.
- Agregar buffers entre etapas para soportar pipeline.
- Implementar una unidad de control capaz de generar señales para instrucciones R, I y J.
- Validar el funcionamiento mediante simulación utilizando TestBench.

4. Desarrollo

Arquitectura Pipeline implementada

La arquitectura desarrollada utiliza cinco etapas principales:

IF (Instruction Fetch): Obtiene la instrucción desde memoria usando el valor actual del contador de programa.

ID (Instruction Decode): Decodifica la instrucción y obtiene los operandos necesarios desde el banco de registros.

EX (Execute): Realiza operaciones aritméticas y lógicas mediante la ALU.

MEM (Memory Access): Permite lectura y escritura sobre memoria de datos.

WB (Write Back): Escribe el resultado obtenido nuevamente al banco de registros.

Y para poder segmentar esta parte se agregaron buffers entre estas etapas, donde estos buffers almacenan señales de control y datos durante cada flanco de subida del reloj.

Implementación de buffers

El funcionamiento de estos buffers se realiza mediante registros secuenciales activados por flanco positivo del reloj. Cuando ocurre un flanco de subida, la información presente en las entradas es almacenada y enviada hacia las salidas correspondientes, manteniendo dichos valores constantes durante el resto del ciclo.

La utilización de estos registros intermedios resulta importante debido a que evita conflictos entre etapas y permite mantener sincronización en el procesamiento. Sin la existencia de estos buffers, las señales podrían sobrescribirse antes de ser utilizadas por las etapas posteriores, ocasionando errores durante la ejecución del procesador. Además de almacenar datos, los buffers también transportan señales de control generadas por la Unidad de Control. Esto permite que cada instrucción conserve sus parámetros durante su recorrido a través del pipeline.

Implementación de instrucciones tipo J

Al agregar el soporte para instrucciones tipo J al MIPS permiten alterar el flujo secuencial de ejecución del programa realizando saltos hacia direcciones específicas de memoria.

Para implementar esta funcionalidad se incorporó una señal de control denominada **Jump**, la cual es generada por la Unidad de Control cuando se detecta el opcode correspondiente a una instrucción de salto. Con esto se tuvo que implementar una lógica adicional para calcular la dirección efectiva de salto utilizando la concatenación de los cuatro bits más significativos de PC+4 con los veintiséis bits correspondientes a la dirección contenida en la instrucción, agregando dos bits en cero al final.

Implementación del algoritmo en ensamblador

El algoritmo seleccionado para ser ejecutado por la arquitectura MIPS fue MergeSort. Debido a que el proyecto trabaja con un conjunto reducido de instrucciones, fue necesario adaptar el algoritmo para utilizar únicamente operaciones soportadas por el datapath implementado.

La implementación del algoritmo requirió operaciones aritméticas para manejo de índices, acceso a memoria para almacenamiento temporal de datos y el uso de instrucciones de salto para controlar el flujo de ejecución. Las instrucciones tipo J fueron utilizadas para redireccionar la ejecución hacia diferentes secciones del programa dependiendo del estado actual del algoritmo.

Durante la ejecución se emplearon registros para almacenar variables temporales, direcciones base y resultados intermedios necesarios durante el proceso de ordenamiento.

Descripción de instrucciones utilizadas

Para la implementación del algoritmo MergeSort dentro del procesador MIPS desarrollado se utilizaron distintas instrucciones tipo R, I y J soportadas por el datapath implementado. Cada una cumple una función específica durante el proceso de ordenamiento y control del flujo del programa.

addi (Add Immediate)

Esta instrucción permite sumar un valor inmediato a un registro y almacenar el resultado en otro registro. Se utilizó para inicializar variables, cargar valores constantes y actualizar índices dentro del algoritmo.

Ejemplo: addi \$t0, \$zero, 5 - Carga el valor 5 en el registro \$t0.

lw (Load Word)

Permite leer una palabra almacenada en memoria y cargarla en un registro. Fue utilizada para recuperar los elementos del arreglo y compararlos durante el proceso de ordenamiento.

Ejemplo: lw \$t0, 0(\$zero) - Carga el contenido almacenado en la posición de memoria indicada al registro \$t0.

sw (Store Word)

Permite almacenar el contenido de un registro dentro de una posición de memoria. Se utilizó para guardar elementos del arreglo y almacenar resultados temporales durante la etapa de mezcla.

Ejemplo: sw \$t0, 0(\$zero) - Guarda el valor contenido en **\$t0** dentro de memoria.

slt (Set Less Than)

Compara dos registros y asigna el valor 1 si el primer operando es menor que el segundo; en caso contrario asigna 0. Esta instrucción se utilizó para determinar el orden entre elementos.

Ejemplo: slt \$t2, \$t1, \$t0 - Si \$t1 es menor que \$t0, entonces \$t2 recibe el valor 1.

slti (Set Less Than Immediate)

Realiza una comparación entre un registro y un valor inmediato. Se utilizó para controlar límites y condiciones dentro de los ciclos.

Ejemplo: slti \$t3, \$t0, 2 - Verifica si el valor de \$t0 es menor que 2.

beq (Branch Equal)

Permite realizar una bifurcación cuando dos registros contienen el mismo valor. Fue utilizada para implementar condiciones y controlar el flujo del algoritmo.

Ejemplo: beq \$t2, \$zero, s23 - Si \$t2 es igual a cero, la ejecución salta a la etiqueta s23.

j (Jump)

Realiza un salto incondicional hacia una dirección específica del programa. Esta instrucción fue implementada durante esta fase para modificar el flujo de ejecución.

Ejemplo: j ml - La ejecución continúa en la etiqueta ml.

Set de instrucciones — Procesador MIPS Pipeline

14 instrucciones soportadas, derivadas de UNITCTRL.v, ALUCTRL.v y ALUMIPS.v.

1. Tabla de instrucciones (ISA)

Tipo	Opcode	Func	Nombre	Sintaxis	Operacion	AluOp	SeIALU
R	000000	100000	add	add \$rd, \$rs, \$rt	$\$rd = \$rs + \$rt$	010	0010 (ADD)
R	000000	100010	sub	sub \$rd, \$rs, \$rt	$\$rd = \$rs - \$rt$	010	0110 (SUB)
R	000000	100100	and	and \$rd, \$rs, \$rt	$\$rd = \$rs \& \$rt$	010	0000 (AND)
R	000000	100101	or	or \$rd, \$rs, \$rt	$\$rd = \$rs \$rt$	010	0001 (OR)
R	000000	101010	slt	slt \$rd, \$rs, \$rt	$\$rd = (\$rs < \$rt) ? 1 : 0$	010	0111 (SLT)
R	000000	100111	nor	nor \$rd, \$rs, \$rt	$\$rd = \sim(\$rs \$rt)$	010	1100 (NOR)
I	100011	—	lw	lw \$rt, imm(\$rs)	$\$rt = \text{MEM}[\$rs + \text{imm}]$	000	0010 (ADD)
I	101011	—	sw	sw \$rt, imm(\$rs)	$\text{MEM}[\$rs + \text{imm}] = \rt	000	0010 (ADD)
I	001000	—	addi	addi \$rt, \$rs, imm	$\$rt = \$rs + \text{SignExt}(\text{imm})$	011	0010 (ADD)
I	001010	—	slti	slti \$rt, \$rs, imm	$\$rt = (\$rs < \text{SignExt}(\text{imm})) ? 1 : 0$	100	0111 (SLT)
I	001101	—	ori	ori \$rt, \$rs, imm	$\$rt = \$rs \text{ZeroExt}(\text{imm})$	101	0001 (OR)
I	001100	—	andi	andi \$rt, \$rs, imm	$\$rt = \$rs \& \text{ZeroExt}(\text{imm})$	110	0000 (AND)
I	000100	—	beq	beq \$rs, \$rt, offset	if $(\$rs == \$rt)$ $\text{PC} = \text{PC} + 4 + (\text{off} < 2)$	001	0110 (SUB)
J	000010	—	j	j target	$\text{PC} = \{\text{PC}[31:28], \text{target}, 00\}$	—	—

2. Señales de control por instrucción

Instrucción	RegDst	AluSrc	MemToReg	RegWrite	MemRead	MemWrite	Branch	Jump
add/sub/and/or/slt/nor	1	0	0	1	0	0	0	0
lw	0	1	1	1	1	0	0	0
sw	0	1	0	0	0	1	0	0
addi/slti	0	1	0	1	0	0	0	0
ori/andi	0	1	0	1	0	0	0	0
beq	0	0	0	0	0	0	1	0
j	0	0	0	0	0	0	0	1

Notas: AluOp se codifica en 3 bits y es interpretado por ALUCTRL para generar SelALU (4 bits). ori y andi usan ZeroExtend para el inmediato (ImmSel=1); el resto usan SignExtend (ImmSel=0). El forwarding y la detección de hazards son manejados por ForwardingUnit.v y HazardUnit.v.

5. Conclusión

Durante el desarrollo de esta fase fue posible comprender de mejor manera el funcionamiento interno de una arquitectura MIPS y la forma en que se realiza el procesamiento de instrucciones mediante un datapath segmentado.

La incorporación de instrucciones tipo J permitió entender cómo se realizan cambios en el flujo normal de ejecución y cómo la dirección de salto es calculada utilizando diferentes partes de la instrucción y del contador de programa.

También se observó la importancia del pipeline y de los buffers entre etapas, ya que permiten mejorar el rendimiento del procesador al ejecutar múltiples instrucciones simultáneamente.

En general, el proyecto permitió aplicar conceptos teóricos vistos durante la materia y comprender de manera más práctica el diseño de procesadores utilizando Verilog.

6. Referencias

Patterson, D. A., & Hennessy, J. L. (2014). Computer Organization and Design: The Hardware/Software Interface (5.^a ed.). Morgan Kaufmann Publishers.

Ciletti, M. D. (2011). Advanced Digital Design with the Verilog HDL (2.^a ed.). Pearson/Prentice Hall.

MIPS Technologies. (2005). MIPS32 Architecture for Programmers, Volume II: The MIPS32 Instruction Set. MIPS Technologies Inc.

Harris, D. M., & Harris, S. L. (2012). Digital Design and Computer Architecture (2.^a ed.). Morgan Kaufmann.